

Laporan Project Fresh Flower Shop

iii. Desain Class

Aplikasi ini memiliki arsitektur CLI terpusat dengan pembagian paket untuk model, dao, database, dan struktur data.

- **freshflowerfinal.Main**: Kelas utama (entry point) dan controller dari seluruh alur program (CLI). Menampung state session user yang sedang login (Customer) dan transaksi yang sedang berjalan (Transaksi), serta memfasilitasi menu untuk Guest, Member, Admin, dan Owner.
- **Model (freshflowerfinal.model)**:
 - **Admin & Owner**: Model untuk representasi kredensial login role admin dan owner.
 - **Customer**: Representasi pelanggan toko bunga (id, nama, password, status member).
 - **Bucket, Bunga, Boneka, Jajan**: Hierarki inheritance (pewarisan). Bucket merupakan parent class yang diturunkan menjadi tipe-tipe produk spesifik.
 - **Transaksi**: Mewakili satu entitas transaksi belanja. Di dalamnya memuat `StackKeranjang`.
- **DAO (freshflowerfinal.dao)**: Data Access Object yang berfungsi melakukan query ke MySQL. Meliputi `BucketDAO`, `CustomerDAO`, dan `TransaksiDAO`.
- **Struktur Data (freshflowerfinal.datastructure)**:
 - **StackKeranjang & NodeKeranjang**: Implementasi LIFO (Last-In First-Out) untuk mengatur keranjang belanja, memfasilitasi fitur *Undo* (menghapus item terakhir yang ditambahkan).
 - **QueueTransaksi & NodeTransaksi**: Implementasi FIFO (First-In First-Out) untuk antrean di sisi Kasir (Admin). Pesanan yang lebih dulu checkout akan diproses pertama kali.
- **Database (freshflowerfinal.database.KoneksiDB)**: Kelas utilitas penyedia koneksi JDBC ke MySQL.

iv. Penjelasan Program

Program Fresh Flower Shop ini telah dikonversi sepenuhnya menjadi aplikasi berbasis Console / Terminal (CLI). Berikut adalah penjelasan proses utama dari sub-menu yang dibuat beserta potongan kodenya:

1. Login dan Guest Registration

Pengguna dapat memilih masuk sebagai Guest (pembeli tamu) atau Member. Jika memilih Guest, program akan membuat ID secara otomatis menggunakan `System.currentTimeMillis()` dan mendaftarkan data sementara di memori serta database. Jika memilih Member, kredensial (ID & Password) akan diuji validitasnya pada database melalui `CustomerDAO.loginMember()`.

2. Katalog Barang dan Tambah ke Keranjang

Menu ini memanggil `bucketDAO.getAllBucket()` untuk menarik list daftar bunga dari database ke array list, kemudian ditampilkan ke layar terminal dengan perulangan `foreach` dan diprint dengan string formatter agar rapi. Saat pembeli menambahkan barang, metode `activeTransaksi.tambahKeKeranjang()` akan mengeksekusi proses Stack `push()` pada item baru, dan `bucketDAO.kurangiStok()` agar stok real di database langsung terupdate.

3. Undo (Hapus Barang Terakhir)

Apabila pelanggan tidak sengaja memasukkan barang, ia dapat memilih menu *Undo*. Menu ini memanfaatkan `activeTransaksi.getKeranjang().pop()` untuk mengeluarkan Node terakhir dari Stack. Jumlah pada node tersebut akan dikembalikan lagi ke database melalui `bucketDAO.kurangiStok(..., -jumlah)`.

4. Kasir dan Antrean (Queue)

Bila pelanggan sudah mengkonfirmasi pembayaran, data Transaksi masuk ke dalam tabel database. Di panel Admin (Kasir), admin akan melihat "Antrean Pesanan". Admin dapat melakukan proses Transaksi dengan memilih menu "Proses Transaksi Pertama", yang menggunakan implementasi algoritma antrean `QueueTransaksi`.

5. Panel Owner

Owner mempunyai akses untuk mengubah Harga Barang yang secara langsung melakukan query Update melalui `BucketDAO`. Owner juga bisa memantau "Laporan Akumulasi Pendapatan Barang" yang datanya diagregasi dari tabel keranjang, serta melihat Grafik Penjualan sederhana berbasis console string builder ('X' merepresentasikan Rp10.000).

v. Source Code

Di bawah ini merupakan implementasi class-class provider, controller utama, beserta class model. Sesuai dengan instruksi, font code di-set dengan ukuran 10.

File: dao\BucketDAO.java

```
package freshflowerfinal.dao;

import freshflowerfinal.database.KoneksiDB;
import freshflowerfinal.model.Bucket;
import freshflowerfinal.model.Bunga;
import freshflowerfinal.model.Boneka;
import freshflowerfinal.model.Jajan;

import java.sql.*;
import java.util.ArrayList;
import java.util.List;

public class BucketDAO {

    public void tambahBucket(Bucket bucket, String jenis) {
        String query = "INSERT INTO bucket (nama, jenis, harga, stok) VALUES (?, ?, ?, ?)";
        try (Connection conn = KoneksiDB.getConnection();
            PreparedStatement stmt = conn.prepareStatement(query)) {
            stmt.setString(1, bucket.getNama());
            stmt.setString(2, jenis);
            stmt.setDouble(3, bucket.getHarga());
            stmt.setInt(4, bucket.getStok());
            stmt.executeUpdate();
        } catch (SQLException e) {
            e.printStackTrace();
        }
    }

    public List<Bucket> getAllBucket() {
        List<Bucket> buckets = new ArrayList<>();
        String query = "SELECT * FROM bucket";
        try (Connection conn = KoneksiDB.getConnection();
            Statement stmt = conn.createStatement();
            ResultSet rs = stmt.executeQuery(query)) {
            while (rs.next()) {
                int idBucket = rs.getInt("id_bucket");
                String nama = rs.getString("nama");
                String jenis = rs.getString("jenis");
                double harga = rs.getDouble("harga");
                int stok = rs.getInt("stok");

                Bucket bucket = null;
                if ("Bunga".equals(jenis)) {
                    bucket = new Bunga(harga, stok, nama);
                } else if ("Boneka".equals(jenis)) {
                    bucket = new Boneka(harga, stok, nama);
                } else if ("Jajan".equals(jenis)) {
                    bucket = new Jajan(harga, stok, nama);
                }

                if (bucket != null) {
                    bucket.setIdBucket(idBucket);
                    buckets.add(bucket);
                }
            }
        } catch (SQLException e) {
            e.printStackTrace();
        }
    }
}
```

```

        return buckets;
    }

    public void ubahBucket(Bucket bucket) {
        String query = "UPDATE bucket SET nama = ?, harga = ?, stok = ? WHERE id_bucket = ?";
        try (Connection conn = KoneksiDB.getConnection();
            PreparedStatement stmt = conn.prepareStatement(query)) {
            stmt.setString(1, bucket.getNama());
            stmt.setDouble(2, bucket.getHarga());
            stmt.setInt(3, bucket.getStok());
            stmt.setInt(4, bucket.getIdBucket());
            stmt.executeUpdate();
        } catch (SQLException e) {
            e.printStackTrace();
        }
    }

    public void hapusBucket(int idBucket) {
        String query = "DELETE FROM bucket WHERE id_bucket = ?";
        try (Connection conn = KoneksiDB.getConnection();
            PreparedStatement stmt = conn.prepareStatement(query)) {
            stmt.setInt(1, idBucket);
            stmt.executeUpdate();
        } catch (SQLException e) {
            e.printStackTrace();
        }
    }

    public void kurangiStok(int idBucket, int jumlahKurang) {
        String query = "UPDATE bucket SET stok = stok - ? WHERE id_bucket = ?";
        try (Connection conn = KoneksiDB.getConnection();
            PreparedStatement stmt = conn.prepareStatement(query)) {
            stmt.setInt(1, jumlahKurang);
            stmt.setInt(2, idBucket);
            stmt.executeUpdate();
        } catch (SQLException e) {
            e.printStackTrace();
        }
    }
}

```

File: dao\CustomerDAO.java

```

package freshflowerfinal.dao;

import freshflowerfinal.database.KoneksiDB;
import freshflowerfinal.model.Customer;

import java.sql.*;
import java.util.ArrayList;
import java.util.List;

public class CustomerDAO {

    public void tambahCustomer(Customer customer) {
        String query = "INSERT INTO customer (id_customer, nama, alamat, nomor_telepon, is_member, password) VALUES (?, ?, ?, ?, ?, ?)";
        try (Connection conn = KoneksiDB.getConnection();
            PreparedStatement stmt = conn.prepareStatement(query)) {
            stmt.setString(1, customer.getIdCustomer());
            stmt.setString(2, customer.getNama());
            stmt.setString(3, customer.getAlamat());
            stmt.setString(4, customer.getNomorTelepon());
            stmt.setBoolean(5, customer.isMember());
            stmt.setString(6, customer.getPassword());
            stmt.executeUpdate();
        } catch (SQLException e) {

```

```

        e.printStackTrace();
    }
}

public List<Customer> getAllCustomer() {
    List<Customer> customers = new ArrayList<>();
    String query = "SELECT * FROM customer";
    try (Connection conn = KoneksiDB.getConnection();
        Statement stmt = conn.createStatement();
        ResultSet rs = stmt.executeQuery(query)) {
        while (rs.next()) {
            customers.add(new Customer(
                rs.getString("id_customer"),
                rs.getString("nama"),
                rs.getString("alamat"),
                rs.getString("nomor_telepon"),
                rs.getBoolean("is_member"),
                rs.getString("password")
            ));
        }
    } catch (SQLException e) {
        e.printStackTrace();
    }
    return customers;
}

public Customer loginMember(String idCustomer, String password) {
    String query = "SELECT * FROM customer WHERE id_customer = ? AND password = ? AND is_member = TRUE";
    try (Connection conn = KoneksiDB.getConnection();
        PreparedStatement stmt = conn.prepareStatement(query)) {
        stmt.setString(1, idCustomer);
        stmt.setString(2, password);
        try (ResultSet rs = stmt.executeQuery()) {
            if (rs.next()) {
                return new Customer(
                    rs.getString("id_customer"),
                    rs.getString("nama"),
                    rs.getString("alamat"),
                    rs.getString("nomor_telepon"),
                    rs.getBoolean("is_member"),
                    rs.getString("password")
                );
            }
        }
    } catch (SQLException e) {
        e.printStackTrace();
    }
    return null;
}

public void ubahPassword(String idCustomer, String newPassword) {
    String query = "UPDATE customer SET password = ? WHERE id_customer = ?";
    try (Connection conn = KoneksiDB.getConnection();
        PreparedStatement stmt = conn.prepareStatement(query)) {
        stmt.setString(1, newPassword);
        stmt.setString(2, idCustomer);
        stmt.executeUpdate();
    } catch (SQLException e) {
        e.printStackTrace();
    }
}

public void ubahCustomer(Customer customer) {
    String query = "UPDATE customer SET nama = ?, alamat = ?, nomor_telepon = ? WHERE id_customer = ?";
    try (Connection conn = KoneksiDB.getConnection();
        PreparedStatement stmt = conn.prepareStatement(query)) {
        stmt.setString(1, customer.getNama());
        stmt.setString(2, customer.getAlamat());
        stmt.setString(3, customer.getNomorTelepon());
        stmt.setString(4, customer.getIdCustomer());
        stmt.executeUpdate();
    } catch (SQLException e) {

```

```

        e.printStackTrace();
    }
}

public void hapusCustomer(String idCustomer) {
    String query = "DELETE FROM customer WHERE id_customer = ?";
    try (Connection conn = KoneksiDB.getConnection();
        PreparedStatement stmt = conn.prepareStatement(query)) {
        stmt.setString(1, idCustomer);
        stmt.executeUpdate();
    } catch (SQLException e) {
        e.printStackTrace();
    }
}
}
}

```

File: dao\TransaksiDAO.java

```

package freshflowerfinal.dao;

import freshflowerfinal.database.KoneksiDB;
import freshflowerfinal.model.Transaksi;
import freshflowerfinal.model.Customer;
import freshflowerfinal.datastructure.StackKeranjang;
import freshflowerfinal.datastructure.NodeKeranjang;

import java.sql.*;
import java.util.ArrayList;
import java.util.List;

public class TransaksiDAO {

    public int simpanTransaksi(Transaksi transaksi, StackKeranjang keranjang) {
        String queryTransaksi = "INSERT INTO transaksi (id_transaksi, id_customer, total_bayar, sudah_dibayar, status_pesanan) VALUES (?, ?, ?, ?, ?)";
        String queryDetail = "INSERT INTO detail_transaksi (id_transaksi, id_bucket, jumlah, total_harga) VALUES (?, ?, ?, ?)";

        Connection conn = null;
        try {
            conn = KoneksiDB.getConnection();
            conn.setAutoCommit(false); // Begin transaction

            // Insert Transaksi
            try (PreparedStatement stmt = conn.prepareStatement(queryTransaksi)) {
                stmt.setInt(1, transaksi.getIdTransaksi());
                stmt.setString(2, transaksi.getCustomer().getIdCustomer());
                stmt.setDouble(3, transaksi.getTotalBayar());
                stmt.setBoolean(4, transaksi.isSudahDibayar());
                stmt.setString(5, "Belum Diproses");
                stmt.executeUpdate();
            }

            // Insert Detail Transaksi
            try (PreparedStatement stmtDetail = conn.prepareStatement(queryDetail)) {
                NodeKeranjang current = keranjang.peek();
                while (current != null) {
                    stmtDetail.setInt(1, transaksi.getIdTransaksi());
                    stmtDetail.setInt(2, current.getBucket().getIdBucket());
                    stmtDetail.setInt(3, current.getJumlah());
                    stmtDetail.setDouble(4, current.getTotalHarga());
                    stmtDetail.addBatch();
                    current = current.getNext();
                }
                stmtDetail.executeBatch();
            }
        } catch (SQLException e) {
            e.printStackTrace();
            if (conn != null) {
                try {
                    conn.rollback();
                } catch (SQLException e2) {
                    e2.printStackTrace();
                }
            }
        } finally {
            if (conn != null) {
                try {
                    conn.close();
                } catch (SQLException e2) {
                    e2.printStackTrace();
                }
            }
        }
    }
}

```

```

        conn.commit();
        return 1;
    } catch (SQLException e) {
        if (conn != null) {
            try { conn.rollback(); } catch (SQLException ex) { ex.printStackTrace(); }
        }
        e.printStackTrace();
        return 0;
    } finally {
        if (conn != null) {
            try { conn.setAutoCommit(true); conn.close(); } catch (SQLException e) { e.printStackTrace(); }
        }
    }
}

public List<Transaksi> getAntreanPesanan() {
    return getTransaksiByStatus("Belum Diproses");
}

public List<Transaksi> getAllTransaksi() {
    return getTransaksiByStatus(null);
}

public void prosesPesanan(int idTransaksi) {
    String query = "UPDATE transaksi SET status_pesanan = 'Selesai' WHERE id_transaksi = ?";
    try (Connection conn = KoneksiDB.getConnection();
        PreparedStatement stmt = conn.prepareStatement(query)) {
        stmt.setInt(1, idTransaksi);
        stmt.executeUpdate();
    } catch (SQLException e) {
        e.printStackTrace();
    }
}

private List<Transaksi> getTransaksiByStatus(String statusFilter) {
    List<Transaksi> list = new ArrayList<>();
    String query = "SELECT t.*, c.nama as cust_nama, c.alamat, c.nomor_telepon, c.is_member, c.password " +
        "FROM transaksi t " +
        "JOIN customer c ON t.id_customer = c.id_customer ";
    if (statusFilter != null) {
        query += "WHERE t.status_pesanan = ?";
    }

    try (Connection conn = KoneksiDB.getConnection();
        PreparedStatement stmt = conn.prepareStatement(query)) {
        if (statusFilter != null) {
            stmt.setString(1, statusFilter);
        }
        try (ResultSet rs = stmt.executeQuery()) {
            while (rs.next()) {
                int idTransaksi = rs.getInt("id_transaksi");
                String idCustomer = rs.getString("id_customer");
                String custNama = rs.getString("cust_nama");
                String alamat = rs.getString("alamat");
                String noTelp = rs.getString("nomor_telepon");
                boolean isMember = rs.getBoolean("is_member");
                String password = rs.getString("password");

                Customer customer = new Customer(idCustomer, custNama, alamat, noTelp, isMember, password);
                Transaksi t = new Transaksi(idTransaksi, customer, 99);
                t.setTotalBayar(rs.getDouble("total_bayar"));
                t.setSudahDibayar(rs.getBoolean("sudah_dibayar"));
                list.add(t);
            }
        }
    } catch (SQLException e) {
        e.printStackTrace();
    }
    return list;
}

public void cetakLaporanPendapatanBarang() {

```

```

String query = "SELECT b.nama, SUM(d.total_harga) as total " +
    "FROM detail_transaksi d " +
    "JOIN bucket b ON d.id_bucket = b.id_bucket " +
    "JOIN transaksi t ON t.id_transaksi = d.id_transaksi " +
    "WHERE t.status_pesanan = 'Selesai' " +
    "GROUP BY b.id_bucket";

String queryTotal = "SELECT SUM(total_bayar) as total_pendapatan FROM transaksi WHERE status_pesanan = 'Selesai'";

try (Connection conn = KoneksiDB.getConnection();
    Statement stmt = conn.createStatement()) {

    double totalPendapatan = 0;
    ResultSet rsTotal = stmt.executeQuery(queryTotal);
    if (rsTotal.next()) {
        totalPendapatan = rsTotal.getDouble("total_pendapatan");
    }
    System.out.println("Total pendapatan : " + totalPendapatan);

    ResultSet rs = stmt.executeQuery(query);
    int i = 1;
    while (rs.next()) {
        String namaBarang = rs.getString("nama");
        double totalSales = rs.getDouble("total");
        System.out.println(i + ". " + namaBarang + " : " + totalSales);
        i++;
    }
} catch (SQLException e) {
    e.printStackTrace();
}

}

public void cetakGrafikPenjualanBarang() {
    String query = "SELECT b.nama, SUM(d.total_harga) as total " +
        "FROM detail_transaksi d " +
        "JOIN bucket b ON d.id_bucket = b.id_bucket " +
        "JOIN transaksi t ON t.id_transaksi = d.id_transaksi " +
        "WHERE t.status_pesanan = 'Selesai' " +
        "GROUP BY b.id_bucket";

    System.out.println("== Grafik Penjualan (Puluhan Ribu) ==");
    try (Connection conn = KoneksiDB.getConnection();
        Statement stmt = conn.createStatement();
        ResultSet rs = stmt.executeQuery(query)) {

        while (rs.next()) {
            String namaBarang = rs.getString("nama");
            double totalSales = rs.getDouble("total");

            int numX = (int) (totalSales / 10000);
            StringBuilder sb = new StringBuilder();
            for (int j = 0; j < numX; j++) {
                sb.append("X");
            }

            System.out.printf("%-10s : %s %.1f\n", namaBarang, sb.toString(), totalSales);
        }
    } catch (SQLException e) {
        e.printStackTrace();
    }
}

}

public void cetakLaporanBelanjaMember() {
    String query = "SELECT c.nama, SUM(t.total_bayar) as total " +
        "FROM transaksi t " +
        "JOIN customer c ON t.id_customer = c.id_customer " +
        "WHERE c.is_member = TRUE AND t.status_pesanan = 'Selesai' " +
        "GROUP BY c.id_customer";

    System.out.println("== Total Belanja per Member ==");
    try (Connection conn = KoneksiDB.getConnection();
        Statement stmt = conn.createStatement());

```

```

        ResultSet rs = stmt.executeQuery(query)) {

    int i = 1;
    boolean found = false;
    while (rs.next()) {
        found = true;
        String namaMember = rs.getString("nama");
        double totalBelanja = rs.getDouble("total");
        System.out.println(i + ". " + namaMember + " : " + totalBelanja);
        i++;
    }
    if (!found) {
        System.out.println("Belum ada data transaksi member.");
    }
} catch (SQLException e) {
    e.printStackTrace();
}

}

public double getTotalPendapatanSelesai() {
    String queryTotal = "SELECT SUM(total_bayar) as total_pendapatan FROM transaksi WHERE status_pesanan = 'Selesai'";
    try (Connection conn = KoneksiDB.getConnection();
        Statement stmt = conn.createStatement();
        ResultSet rs = stmt.executeQuery(queryTotal)) {
        if (rs.next()) {
            return rs.getDouble("total_pendapatan");
        }
    } catch (SQLException e) {
        e.printStackTrace();
    }
    return 0;
}

public static class LaporanBarang {
    private String nama;
    private double total;
    public LaporanBarang(String nama, double total) {
        this.nama = nama;
        this.total = total;
    }
    public String getNama() { return nama; }
    public double getTotal() { return total; }
}

public static class LaporanMember {
    private String nama;
    private double total;
    public LaporanMember(String nama, double total) {
        this.nama = nama;
        this.total = total;
    }
    public String getNama() { return nama; }
    public double getTotal() { return total; }
}

public List<LaporanBarang> getLaporanPendapatanBarangList() {
    List<LaporanBarang> list = new ArrayList<>();
    String query = "SELECT b.nama, SUM(d.total_harga) as total " +
        "FROM detail_transaksi d " +
        "JOIN bucket b ON d.id_bucket = b.id_bucket " +
        "JOIN transaksi t ON t.id_transaksi = d.id_transaksi " +
        "WHERE t.status_pesanan = 'Selesai' " +
        "GROUP BY b.id_bucket";
    try (Connection conn = KoneksiDB.getConnection();
        Statement stmt = conn.createStatement();
        ResultSet rs = stmt.executeQuery(query)) {
        while (rs.next()) {
            list.add(new LaporanBarang(rs.getString("nama"), rs.getDouble("total")));
        }
    } catch (SQLException e) {
        e.printStackTrace();
    }
}

```

```

        return list;
    }

    public List<LaporanMember> getLaporanBelanjaMemberList() {
        List<LaporanMember> list = new ArrayList<>();
        String query = "SELECT c.nama, SUM(t.total_bayar) as total " +
            "FROM transaksi t " +
            "JOIN customer c ON t.id_customer = c.id_customer " +
            "WHERE c.is_member = TRUE AND t.status_pesanan = 'Selesai' " +
            "GROUP BY c.id_customer";
        try (Connection conn = KoneksiDB.getConnection();
            Statement stmt = conn.createStatement();
            ResultSet rs = stmt.executeQuery(query)) {
            while (rs.next()) {
                list.add(new LaporanMember(rs.getString("nama"), rs.getDouble("total")));
            }
        } catch (SQLException e) {
            e.printStackTrace();
        }
        return list;
    }
}

```

File: database\KoneksiDB.java

```

package freshflowerfinal.database;

import java.sql.Connection;
import java.sql.DriverManager;

public class KoneksiDB {
    private static final String URL = "jdbc:mysql://localhost:3306/freshflower_db";
    private static final String USER = "root";
    private static final String PASSWORD = "";

    public static Connection getConnection() {
        try {
            Class.forName("com.mysql.cj.jdbc.Driver");
            return DriverManager.getConnection(URL, USER, PASSWORD);
        } catch (Exception e) {
            System.out.println("Gagal terhubung ke database!");
            e.printStackTrace();
            return null;
        }
    }
}

```

File: datastructure\NodeKeranjang.java

```

package freshflowerfinal.datastructure;

import freshflowerfinal.model.Bucket;

public class NodeKeranjang {
    private Bucket bucket;
    private int jumlah;
    private double totalHarga;
    private NodeKeranjang next;

    public NodeKeranjang(Bucket bucket, int jumlah) {
        this.bucket = bucket;
    }
}

```

```

        this.jumlah = jumlah;
        this.totalHarga = bucket.getHarga() * jumlah;
        this.next = null;
    }

    public Bucket getBucket() { return bucket; }
    public int getJumlah() { return jumlah; }
    public double getTotalHarga() { return totalHarga; }
    public NodeKeranjang getNext() { return next; }
    public void setNext(NodeKeranjang next) { this.next = next; }
}

```

File: datastructure\NodeTransaksi.java

```

package freshflowerfinal.datastructure;

import freshflowerfinal.model.Transaksi;

public class NodeTransaksi {
    private Transaksi transaksi;
    private NodeTransaksi next;

    public NodeTransaksi(Transaksi transaksi) {
        this.transaksi = transaksi;
        this.next = null;
    }

    public Transaksi getTransaksi() { return transaksi; }
    public NodeTransaksi getNext() { return next; }
    public void setNext(NodeTransaksi next) { this.next = next; }
}

```

File: datastructure\QueueTransaksi.java

```

package freshflowerfinal.datastructure;

import freshflowerfinal.model.Transaksi;

public class QueueTransaksi {
    private NodeTransaksi front;
    private NodeTransaksi rear;
    private int size;

    public QueueTransaksi() {
        this.front = null;
        this.rear = null;
        this.size = 0;
    }

    public boolean isEmpty() {
        return front == null;
    }

    public void enqueue(Transaksi transaksi) {
        NodeTransaksi newNode = new NodeTransaksi(transaksi);
        if (isEmpty()) {
            front = newNode;
            rear = newNode;
        } else {
            rear.setNext(newNode);
            rear = newNode;
        }
    }
}

```

```

    }
    size++;
}

public Transaksi dequeue() {
    if (isEmpty()) {
        return null;
    }
    Transaksi removedTransaksi = front.getTransaksi();
    front = front.getNext();
    if (front == null) {
        rear = null;
    }
    size--;
    return removedTransaksi;
}

public Transaksi peek() {
    if (isEmpty()) {
        return null;
    }
    return front.getTransaksi();
}

public int getSize() {
    return size;
}

public void tampilkanAntrean() {
    if (isEmpty()) {
        System.out.println("Antrean pesanan kosong.");
        return;
    }
    System.out.println("== Antrean Pesanan ==");
    NodeTransaksi current = front;
    int i = 1;
    while (current != null) {
        System.out.println(i + ". ID Transaksi: " + current.getTransaksi().getIdTransaksi() + " - Customer: " +
current.getTransaksi().getCustomer().getNama());
        current = current.getNext();
        i++;
    }
}
}

```

File: datastructure\StackKeranjang.java

```

package freshflowerfinal.datastructure;

import freshflowerfinal.model.Bucket;

public class StackKeranjang {
    private NodeKeranjang top;

    public StackKeranjang() {
        this.top = null;
    }

    public boolean isEmpty() {
        return top == null;
    }

    public void push(Bucket bucket, int jumlah) {
        NodeKeranjang newNode = new NodeKeranjang(bucket, jumlah);
        if (top != null) {
            newNode.setNext(top);
        }
    }
}

```

```

        top = newNode;
    }

    public NodeKeranjang pop() {
        if (isEmpty()) {
            return null;
        }
        NodeKeranjang poppedNode = top;
        top = top.getNext();
        return poppedNode;
    }

    public NodeKeranjang peek() {
        return top;
    }

    public double hitungTotalBayar() {
        double total = 0;
        NodeKeranjang current = top;
        while (current != null) {
            total += current.getTotalHarga();
            current = current.getNext();
        }
        return total;
    }

    public void kurangiStok() {
        NodeKeranjang current = top;
        while (current != null) {
            Bucket b = current.getBucket();
            b.setStok(b.getStok() - current.getJumlah());
            current = current.getNext();
        }
    }

    public int getUkuran() {
        int count = 0;
        NodeKeranjang current = top;
        while (current != null) {
            count++;
            current = current.getNext();
        }
        return count;
    }

    public void tampilkan() {
        if (isEmpty()) {
            System.out.println("Keranjang kosong.");
            return;
        }
        System.out.println("== Keranjang ==");
        NodeKeranjang current = top;
        int i = 1;
        while (current != null) {
            System.out.println(i + ". Nama Barang: " + current.getBucket().getNama());
            System.out.println("    Harga Barang: " + current.getBucket().getHarga());
            System.out.println("    Jumlah: " + current.getJumlah());
            System.out.println("    Total Harga: " + current.getTotalHarga());
            current = current.getNext();
            i++;
        }
    }
}

```

```

package freshflowerfinal;

import freshflowerfinal.database.KoneksiDB;
import java.sql.Connection;
import java.sql.Statement;

public class DBFixer {
    public static void main(String[] args) {
        try (Connection conn = KoneksiDB.getConnection();
            Statement stmt = conn.createStatement()) {

            try {
                stmt.execute("ALTER TABLE customer ADD COLUMN is_member BOOLEAN DEFAULT FALSE");
                System.out.println("Berhasil menambahkan kolom is_member.");
            } catch (Exception e) {
                System.out.println("Kolom is_member mungkin sudah ada.");
            }

            try {
                stmt.execute("ALTER TABLE customer ADD COLUMN password VARCHAR(50) NULL");
                System.out.println("Berhasil menambahkan kolom password.");
            } catch (Exception e) {
                System.out.println("Kolom password mungkin sudah ada.");
            }

            try {
                stmt.executeUpdate("INSERT IGNORE INTO customer (id_customer, nama, alamat, nomor_telepon, is_member, password) VALUES ('2024-001', 'Budi', 'Jakarta', '08111', TRUE, 'member1')");
                stmt.executeUpdate("INSERT IGNORE INTO customer (id_customer, nama, alamat, nomor_telepon, is_member, password) VALUES ('2024-002', 'Wati', 'Surabaya', '08222', TRUE, 'member2')");
                stmt.executeUpdate("INSERT IGNORE INTO customer (id_customer, nama, alamat, nomor_telepon, is_member, password) VALUES ('2024-003', 'Iwan', 'Semarang', '08333', TRUE, 'member3')");
                System.out.println("Berhasil memasukkan data akun Member (2024-001, 2024-002, 2024-003).");
            } catch (Exception e) {
                System.out.println("Gagal memasukkan data member.");
                e.printStackTrace();
            }

            System.out.println("Perbaikan Database Selesai!");
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}

```

File: Main.java

```

package freshflowerfinal;

import freshflowerfinal.dao.*;
import freshflowerfinal.model.*;
import freshflowerfinal.datastructure.NodeKeranjang;
import java.util.Scanner;
import java.util.List;

public class Main {
    // DAOs
    private static final BucketDAO bucketDAO = new BucketDAO();
    private static final CustomerDAO customerDAO = new CustomerDAO();
    private static final TransaksiDAO transaksiDAO = new TransaksiDAO();

    // Authenticated credentials / active session state
    private static final Admin admin = new Admin();
    private static final Owner owner = new Owner();
    private static Customer activeCustomer;
    private static Transaksi activeTransaksi;
}

```

```

private static final Scanner scanner = new Scanner(System.in);

public static void main(String[] args) {
    // Cek koneksi database MySQL
    try (java.sql.Connection conn = freshflowerfinal.database.KoneksiDB.getConnection()) {
        if (conn == null) {
            System.out.println("Gagal menghubungkan ke Database MySQL! Harap pastikan MySQL telah di-Start.");
            System.exit(0);
        }
    } catch (Exception e) {
        System.out.println("Gagal menghubungkan ke Database!");
        System.exit(0);
    }

    while (true) {
        System.out.println("\n=====");
        System.out.println("          * FRESH FLOWER SHOP Menu *          ");
        System.out.println("=====");
        System.out.println("Pilih Role Anda:");
        System.out.println("1. Pembeli (Guest)");
        System.out.println("2. Member");
        System.out.println("3. Admin");
        System.out.println("4. Owner (Pemilik)");
        System.out.println("5. Keluar");
        System.out.print("Masukkan Pilihan [1-5]: ");
        String pilihan = scanner.nextLine().trim();

        switch (pilihan) {
            case "1":
                runPembeliGuestFlow();
                break;
            case "2":
                runMemberFlow();
                break;
            case "3":
                runAdminFlow();
                break;
            case "4":
                runOwnerFlow();
                break;
            case "5":
                System.out.println("Terima kasih telah berkunjung ke toko kami!");
                System.exit(0);
            default:
                System.out.println("Pilihan tidak valid! Masukkan angka antara 1 sampai 5.");
        }
    }
}

private static void runPembeliGuestFlow() {
    System.out.print("Masukkan Nama Anda: ");
    String nama = scanner.nextLine().trim();
    if (nama.isEmpty()) {
        System.out.println("Nama tidak boleh kosong!");
        return;
    }

    // Register Guest
    String idCustomer = "GST" + (System.currentTimeMillis() % 10000);
    activeCustomer = new Customer(idCustomer, nama, "-", "-", false, null);
    customerDAO.tambahCustomer(activeCustomer);

    int newId = (int) (System.currentTimeMillis() / 1000);
    activeTransaksi = new Transaksi(newId, activeCustomer, 99);

    System.out.println("\nSelamat Datang Guest: " + activeCustomer.getNama());
    runShopFlow();
}

private static void runMemberFlow() {
    System.out.print("Masukkan ID Member: ");
}

```

```

String id = scanner.nextLine().trim();
System.out.print("Masukkan Password: ");
String pass = scanner.nextLine().trim();

Customer member = customerDAO.loginMember(id, pass);
if (member == null) {
    System.out.println("ID Member atau Password salah!");
    return;
}
activeCustomer = member;
int newId = (int) (System.currentTimeMillis() / 1000);
activeTransaksi = new Transaksi(newId, activeCustomer, 99);

while (true) {
    System.out.println("\n-----");
    System.out.println("Dashboard Member: " + member.getNama() + " (Diskon 5% Aktif)");
    System.out.println("-----");
    System.out.println("1. Mulai Belanja");
    System.out.println("2. Ubah Password");
    System.out.println("3. Kembali ke Menu Utama");
    System.out.print("Pilih menu [1-3]: ");
    String pilihan = scanner.nextLine().trim();

    if (pilihan.equals("1")) {
        runShopFlow();
    } else if (pilihan.equals("2")) {
        System.out.print("Masukkan password baru: ");
        String newPass = scanner.nextLine().trim();
        if (!newPass.isEmpty()) {
            customerDAO.ubahPassword(activeCustomer.getIdCustomer(), newPass);
            activeCustomer.setPassword(newPass);
            System.out.println("Password berhasil diubah!");
        } else {
            System.out.println("Gagal mengubah password! Password tidak boleh kosong.");
        }
    } else if (pilihan.equals("3")) {
        break;
    } else {
        System.out.println("Pilihan tidak valid!");
    }
}
}

private static void runShopFlow() {
    while (true) {
        System.out.println("\n-----");
        System.out.println("                KATALOG BARANG                ");
        System.out.println("-----");
        List<Bucket> catalog = bucketDAO.getAllBucket();
        System.out.printf("%-5s | %-25s | %-10s | %-12s | %-6s\n", "ID", "Nama Barang", "Jenis", "Harga", "Stok");
        System.out.println("-----");
        for (Bucket b : catalog) {
            String jenis = b instanceof Bunga ? "Bunga" : (b instanceof Boneka ? "Boneka" : "Jajan");
            System.out.printf("%-5d | %-25s | %-10s | Rp %-10.0f | %-6d\n",
                b.getIdBucket(), b.getNama(), jenis, b.getHarga(), b.getStok());
        }
        System.out.println("-----");

        System.out.println("1. Tambah Barang ke Keranjang");
        System.out.println("2. Undo (Hapus Barang Terakhir)");
        System.out.println("3. Lihat Keranjang & Total Bayar");
        System.out.println("4. Selesai & Bayar (Kirim ke Antrean)");
        System.out.println("5. Batal & Kembali");
        System.out.print("Pilih Menu [1-5]: ");
        String menu = scanner.nextLine().trim();

        if (menu.equals("1")) {
            System.out.print("Masukkan ID Barang: ");
            try {
                int idBucket = Integer.parseInt(scanner.nextLine().trim());
                System.out.print("Masukkan Jumlah: ");
                int qty = Integer.parseInt(scanner.nextLine().trim());
            }

```

```

        if (qty <= 0) {
            System.out.println("Jumlah harus lebih dari 0!");
            continue;
        }

        // Add to cart directly
        List<Bucket> buckets = bucketDAO.getAllBucket();
        Bucket target = null;
        for (Bucket b : buckets) {
            if (b.getIdBucket() == idBucket) {
                target = b;
                break;
            }
        }

        if (target != null && qty <= target.getStok()) {
            activeTransaksi.tambahKeKeranjang(target, qty);
            bucketDAO.kurangiStok(idBucket, qty);
            System.out.println("Berhasil menambahkan ke keranjang.");
        } else {
            System.out.println("Gagal menambahkan! Stok tidak mencukupi atau ID salah.");
        }
    } catch (NumberFormatException e) {
        System.out.println("Input harus berupa angka!");
    }
} else if (menu.equals("2")) {
    if (activeTransaksi != null) {
        NodeKeranjang popped = activeTransaksi.getKeranjang().pop();
        if (popped != null) {
            Bucket b = popped.getBucket();
            bucketDAO.kurangiStok(b.getIdBucket(), -popped.getJumlah());

            double totalSem = activeTransaksi.getKeranjang().hitungTotalBayar();
            activeTransaksi.setTotalBayar(totalSem * (activeCustomer.isMember() ? 0.95 : 1.0));
            System.out.println("Berhasil menghapus: " + b.getNama() + " (" + popped.getJumlah() + " pcs)");
        } else {
            System.out.println("Keranjang kosong!");
        }
    } else {
        System.out.println("Keranjang kosong!");
    }
} else if (menu.equals("3")) {
    tampilkanKeranjang();
} else if (menu.equals("4")) {
    if (activeTransaksi != null && activeTransaksi.getUkuranKeranjang() > 0) {
        double total = activeTransaksi.getTotalBayar();
        int status = transaksiDAO.simpanTransaksi(activeTransaksi, activeTransaksi.getKeranjang());
        if (status == 1) {
            System.out.println("Transaksi berhasil disimpan! Total Bayar: Rp " + String.format("%.0f", total));
            System.out.println("Silakan lakukan pembayaran di kasir (Admin).");
            activeTransaksi = null;
            activeCustomer = null;
            break;
        } else {
            System.out.println("Gagal menyimpan transaksi!");
        }
    } else {
        System.out.println("Keranjang belanja masih kosong!");
    }
} else if (menu.equals("5")) {
    if (activeTransaksi != null) {
        NodeKeranjang curr = activeTransaksi.getKeranjang().peek();
        while (curr != null) {
            bucketDAO.kurangiStok(curr.getBucket().getIdBucket(), -curr.getJumlah());
            curr = curr.getNext();
        }
        activeTransaksi = null;
        activeCustomer = null;
    }
    System.out.println("Pembelian dibatalkan.");
    break;
} else {

```

```

        System.out.println("Pilihan tidak valid!");
    }
}

private static void tampilkanKeranjang() {
    if (activeTransaksi == null) {
        System.out.println("Belum ada transaksi aktif!");
        return;
    }
    System.out.println("\n-----");
    System.out.println("                KERANJANG BELANJA                ");
    System.out.println("-----");
    NodeKeranjang curr = activeTransaksi.getKeranjang().peek();
    if (curr == null) {
        System.out.println("(Keranjang Kosong)");
    } else {
        System.out.printf("%-25s | %-8s | %-12s\n", "Nama Barang", "Jumlah", "Total Harga");
        System.out.println("-----");
        while (curr != null) {
            System.out.printf("%-25s | %-8d | Rp %-10.0f\n",
                curr.getBucket().getNama(), curr.getJumlah(), curr.getTotalHarga());
            curr = curr.getNext();
        }
        System.out.println("-----");
        System.out.println("Total Bayar: Rp " + String.format("%.0f", activeTransaksi.getTotalBayar()));
    }
}

private static void runAdminFlow() {
    System.out.print("Masukkan Username Admin: ");
    String user = scanner.nextLine().trim();
    System.out.print("Masukkan Password Admin: ");
    String pass = scanner.nextLine().trim();

    if (!admin.login(user, pass)) {
        System.out.println("Username atau Password Admin salah!");
        return;
    }

    while (true) {
        System.out.println("\n-----");
        System.out.println("                KASIR / ADMIN PANEL                ");
        System.out.println("-----");
        System.out.println("1. Lihat Antrean Transaksi (FIFO)");
        System.out.println("2. Proses Transaksi Pertama");
        System.out.println("3. Kembali ke Menu Utama");
        System.out.print("Pilih Menu [1-3]: ");
        String menu = scanner.nextLine().trim();

        if (menu.equals("1")) {
            List<Transaksi> antrean = transaksiDAO.getAntreanPesanan();
            if (antrean.isEmpty()) {
                System.out.println("(Antrean Kosong)");
            } else {
                System.out.printf("%-15s | %-20s | %-10s | %-12s\n", "ID Transaksi", "Nama Pelanggan", "Status", "Total
Bayar");
                System.out.println("-----");
                for (Transaksi t : antrean) {
                    String statusPelanggan = t.getCustomer().isMember() ? "Member" : "Guest";
                    System.out.printf("%-15d | %-20s | %-10s | Rp %-10.0f\n",
                        t.getIdTransaksi(), t.getCustomer().getNama(), statusPelanggan, t.getTotalBayar());
                }
            }
        } else if (menu.equals("2")) {
            List<Transaksi> antrean = transaksiDAO.getAntreanPesanan();
            if (!antrean.isEmpty()) {
                Transaksi diproses = antrean.get(0);
                transaksiDAO.prosesPesanan(diproses.getIdTransaksi());
                System.out.println("Berhasil memproses Transaksi ID " + diproses.getIdTransaksi() +
                    " atas nama " + diproses.getCustomer().getNama());
            }
        } else {
    }
}

```

```

        System.out.println("Tidak ada antrean transaksi!");
    }
} else if (menu.equals("3")) {
    break;
} else {
    System.out.println("Pilihan tidak valid!");
}
}
}

private static void runOwnerFlow() {
    System.out.print("Masukkan Username Owner: ");
    String user = scanner.nextLine().trim();
    System.out.print("Masukkan Password Owner: ");
    String pass = scanner.nextLine().trim();

    if (!owner.login(user, pass)) {
        System.out.println("Username atau Password Owner salah!");
        return;
    }

    while (true) {
        System.out.println("\n-----");
        System.out.println("OWNER PANEL");
        System.out.println("-----");
        System.out.println("1. Kelola/Update Harga Barang");
        System.out.println("2. Laporan Akumulasi Pendapatan Barang");
        System.out.println("3. Laporan Akumulasi Belanja Member");
        System.out.println("4. Cetak Grafik Penjualan");
        System.out.println("5. Kembali ke Menu Utama");
        System.out.print("Pilih Menu [1-5]: ");
        String menu = scanner.nextLine().trim();

        if (menu.equals("1")) {
            List<Bucket> items = bucketDAO.getAllBucket();
            System.out.printf("%-5s | %-25s | %-12s | %-6s\n", "ID", "Nama Barang", "Harga", "Stok");
            System.out.println("-----");
            for (Bucket b : items) {
                System.out.printf("%-5d | %-25s | Rp %-10.0f | %-6d\n", b.getIdBucket(), b.getNama(), b.getHarga(),
b.getStok());
            }
            System.out.println("-----");
            System.out.print("Masukkan ID Barang yang ingin diubah harganya: ");
            try {
                int idBucket = Integer.parseInt(scanner.nextLine().trim());
                System.out.print("Masukkan Harga Baru (Rp): ");
                double newPrice = Double.parseDouble(scanner.nextLine().trim());
                if (newPrice < 0) {
                    System.out.println("Harga tidak valid!");
                    continue;
                }

                List<Bucket> buckets = bucketDAO.getAllBucket();
                Bucket target = null;
                for (Bucket b : buckets) {
                    if (b.getIdBucket() == idBucket) {
                        target = b;
                        break;
                    }
                }
                if (target != null) {
                    target.setHarga(newPrice);
                    bucketDAO.ubahBucket(target);
                    System.out.println("Harga berhasil diperbarui.");
                } else {
                    System.out.println("Gagal memperbarui! ID tidak ditemukan.");
                }
            } catch (NumberFormatException e) {
                System.out.println("Input harus berupa angka!");
            }
        } else if (menu.equals("2")) {
            System.out.println("\n--- Laporan Pendapatan per Barang ---");

```

```

        List<TransaksiDAO.LaporanBarang> listBarang = transaksiDAO.getLaporanPendapatanBarangList();
        if (listBarang.isEmpty()) {
            System.out.println("Belum ada transaksi.");
        } else {
            int i = 1;
            for (TransaksiDAO.LaporanBarang lb : listBarang) {
                System.out.println(i + ". " + lb.getNama() + " : Rp " + String.format("%.0f", lb.getTotal()));
                i++;
            }
        }
    } else if (menu.equals("3")) {
        System.out.println("\n--- Laporan Belanja Member ---");
        List<TransaksiDAO.LaporanMember> listMember = transaksiDAO.getLaporanBelanjaMemberList();
        if (listMember.isEmpty()) {
            System.out.println("Belum ada transaksi member.");
        } else {
            int i = 1;
            for (TransaksiDAO.LaporanMember lm : listMember) {
                System.out.println(i + ". " + lm.getNama() + " : Rp " + String.format("%.0f", lm.getTotal()));
                i++;
            }
        }
    } else if (menu.equals("4")) {
        System.out.println("\n--- Grafik Penjualan (Setiap X mewakili Rp 10.000) ---");
        List<TransaksiDAO.LaporanBarang> listBarang = transaksiDAO.getLaporanPendapatanBarangList();
        if (listBarang.isEmpty()) {
            System.out.println("Belum ada data penjualan.");
        } else {
            for (TransaksiDAO.LaporanBarang lb : listBarang) {
                int xCount = (int) (lb.getTotal() / 10000);
                StringBuilder sb = new StringBuilder();
                for (int k = 0; k < xCount; k++) {
                    sb.append("X");
                }
                System.out.printf("%-25s : %s (Rp %,0f)\n", lb.getNama(), sb.toString(), lb.getTotal());
            }
        }
    } else if (menu.equals("5")) {
        break;
    } else {
        System.out.println("Pilihan tidak valid!");
    }
}
}
}

```

File: modelAdmin.java

```

package freshflowerfinal.model;

public class Admin {
    private String username = "Admin";
    private String password = "Admin123";

    public Admin() {}

    public String getUsername() {
        return username;
    }

    public void setUsername(String username) {
        this.username = username;
    }

    public String getPassword() {
        return password;
    }
}

```

```

    public void setPassword(String password) {
        this.password = password;
    }

    public boolean login(String username, String password) {
        if (this.username.equals(username) && this.password.equals(password)) {
            return true;
        } else {
            return false;
        }
    }
}

```

File: model\Boneka.java

```

package freshflowerfinal.model;

public class Boneka extends Bucket {
    private String karakter;

    public Boneka() {
        System.out.println("Data Boneka dibuat");
    }

    public Boneka(double harga, int stok, String karakter) {
        super(harga, stok);
        this.karakter = karakter;
        super.setNama(karakter);
    }

    public String getKarakter() {
        return karakter;
    }

    public void setKarakter(String karakter) {
        this.karakter = karakter;
        super.setNama(karakter);
    }

    @Override
    public void setNama(String nama) {
        this.karakter = nama;
        super.setNama(nama);
    }

    @Override
    public String toString() {
        return "Nama: " + getNama() + "\tHarga: " + getHarga() + "\tStok: " + getStok();
    }
}

```

File: model\Bucket.java

```

package freshflowerfinal.model;

public class Bucket {
    private int idBucket;
    private double harga;
    private int stok;
    private String nama;
}

```

```

public int getIdBucket() { return idBucket; }
public void setIdBucket(int idBucket) { this.idBucket = idBucket; }

public Bucket() {
    System.out.println("Data Bucket dibuat");
}

public Bucket(double harga, int stok) {
    this.harga = harga;
    this.stok = stok;
}

public double getHarga() {
    return harga;
}

public void setHarga(double harga) {
    this.harga = harga;
}

public int getStok() {
    return stok;
}

public void setStok(int stok) {
    this.stok = stok;
}

public String getNama() {
    return nama;
}

public void setNama(String nama) {
    this.nama = nama;
}

public String toString() {
    return "Nama: " + nama + "\tHarga: " + harga + "\tStok: " + stok + " ";
}
}

```

File: model\Bunga.java

```

package freshflowerfinal.model;

public class Bunga extends Bucket {
    private String jenis;

    public Bunga() {
        System.out.println("Data Bunga dibuat");
    }

    public Bunga(double harga, int stok, String jenis) {
        super(harga, stok);
        this.jenis = jenis;
        super.setNama(jenis);
    }

    public String getJenis() {
        return jenis;
    }

    public void setJenis(String jenis) {
        this.jenis = jenis;
        super.setNama(jenis);
    }
}

```

```

@Override
public void setNama(String nama) {
    this.jenis = nama;
    super.setNama(nama);
}

@Override
public String toString() {
    return "Nama: " + getNama() + "\tHarga: " + getHarga() + "\tStok: " + getStok();
}
}

```

File: model\Customer.java

```

package freshflowerfinal.model;

public class Customer {
    private String idCustomer;
    private String nama;
    private String alamat;
    private String nomorTelepon;
    private boolean isMember;
    private String password;

    public Customer() {
        System.out.println("Data Customer dibuat");
    }

    public Customer(String idCustomer, String nama, String alamat, String nomorTelepon) {
        this.idCustomer = idCustomer;
        this.nama = nama;
        this.alamat = alamat;
        this.nomorTelepon = nomorTelepon;
        this.isMember = false;
        this.password = null;
    }

    public Customer(String idCustomer, String nama, String alamat, String nomorTelepon, boolean isMember, String password) {
        this.idCustomer = idCustomer;
        this.nama = nama;
        this.alamat = alamat;
        this.nomorTelepon = nomorTelepon;
        this.isMember = isMember;
        this.password = password;
    }

    public String getIdCustomer() {
        return idCustomer;
    }

    public void setIdCustomer(String idCustomer) {
        this.idCustomer = idCustomer;
    }

    public String getNama() {
        return nama;
    }

    public void setNama(String nama) {
        this.nama = nama;
    }

    public String getAlamat() {
        return alamat;
    }
}

```

```

    public void setAlamat(String alamat) {
        this.alamat = alamat;
    }

    public String getNomorTelepon() {
        return nomorTelepon;
    }

    public void setNomorTelepon(String nomorTelepon) {
        this.nomorTelepon = nomorTelepon;
    }

    public boolean isMember() {
        return isMember;
    }

    public void setMember(boolean isMember) {
        this.isMember = isMember;
    }

    public String getPassword() {
        return password;
    }

    public void setPassword(String password) {
        this.password = password;
    }

    @Override
    public String toString() {
        return "ID: " + idCustomer + "\tNama: " + nama + "\tAlamat: " + alamat + "\tTelp: " + nomorTelepon + (isMember ? " "
[MEMBER]" : "");
    }
}

```

File: model\Jajan.java

```

package freshflowerfinal.model;

public class Jajan extends Bucket {
    private String merk;

    public Jajan() {
        System.out.println("Data Jajan dibuat");
    }

    public Jajan(double harga, int stok, String merk) {
        super(harga, stok);
        this.merk = merk;
        super.setNama(merk);
    }

    public String getMerk() {
        return merk;
    }

    public void setMerk(String merk) {
        this.merk = merk;
        super.setNama(merk);
    }

    @Override
    public void setNama(String nama) {
        this.merk = nama;
        super.setNama(nama);
    }
}

```

```

@Override
public String toString() {
    return "Nama: " + getNama() + "\tHarga: " + getHarga() + "\tStok: " + getStok();
}
}

```

File: model\Owner.java

```

package freshflowerfinal.model;

public class Owner {
    private String nama = "Owner";
    private String password = "Owner123";

    public Owner() {}

    public String getNama() {
        return nama;
    }

    public String getPassword() {
        return password;
    }

    public boolean login(String nama, String password) {
        if (this.nama.equals(nama) && this.password.equals(password)) {
            return true;
        } else {
            return false;
        }
    }
}

```

File: model\Transaksi.java

```

package freshflowerfinal.model;

import freshflowerfinal.datastructure.NodeKeranjang;
import freshflowerfinal.datastructure.StackKeranjang;

public class Transaksi {
    private int idTransaksi;
    private Customer customer;
    private StackKeranjang keranjang;
    private double totalBayar;
    private boolean sudahDibayar;

    public Transaksi(int idTransaksi, Customer customer, int kapasitas) {
        this.idTransaksi = idTransaksi;
        this.customer = customer;
        this.keranjang = new StackKeranjang();
        this.totalBayar = 0;
        this.sudahDibayar = false;
    }

    public int getIdTransaksi() {
        return idTransaksi;
    }

    public StackKeranjang getKeranjang() {
        return keranjang;
    }
}

```

```

    }

    public Customer getCustomer() {
        return customer;
    }

    public double getTotalBayar() {
        return totalBayar;
    }

    public void setTotalBayar(double totalBayar) {
        this.totalBayar = totalBayar;
    }

    public boolean isSudahDibayar() {
        return sudahDibayar;
    }

    public void setSudahDibayar(boolean sudahDibayar) {
        this.sudahDibayar = sudahDibayar;
    }

    public int getUkuranKeranjang() {
        return keranjang.getUkuran();
    }

    public boolean tambahKeKeranjang(Bucket b, int jumlahBeli) {
        keranjang.push(b, jumlahBeli);
        hitungTotalDiskon();
        return true;
    }

    public boolean hapusItemTerakhir() {
        NodeKeranjang popped = keranjang.pop();
        if (popped != null) {
            System.out.println("Item " + popped.getBucket().getNama() + " berhasil dihapus (Undo).");
            hitungTotalDiskon();
            return true;
        } else {
            System.out.println("Keranjang kosong, tidak ada yang bisa di-undo.");
            return false;
        }
    }

    private void hitungTotalDiskon() {
        double totalSementara = keranjang.hitungTotalBayar();
        if (customer != null && customer.isMember()) {
            totalBayar = totalSementara * 0.95; // Diskon 5%
        } else {
            totalBayar = totalSementara;
        }
    }

    public void lihatKeranjang() {
        keranjang.tampilkan();
        if (!keranjang.isEmpty()) {
            System.out.println("Total Bayar: " + totalBayar);
        }
    }

    public boolean bayar(double uangBayar) {
        if (uangBayar >= totalBayar) {
            sudahDibayar = true;
            System.out.println("Pembayaran berhasil. Kembalian: Rp" + (uangBayar - totalBayar));
            return true;
        }

        System.out.println("Uang kurang!");
        return false;
    }

    public void kurangiStok() {

```

```
        keranjang.kurangiStok();
    }

    public void cetakRingkasan() {
        System.out.println("ID Transaksi: " + idTransaksi);
        System.out.println("Customer: " + customer.getNama());
        System.out.println("Total Bayar: " + totalBayar);
        System.out.println("Status: " + (sudahDibayar ? "Lunas" : "Belum Dibayar"));
    }

    public void strukTransaksi() {
        System.out.println("=== STRUK TRANSAKSI ===");
        System.out.println("ID Transaksi: " + idTransaksi);
        System.out.println("ID Customer: " + customer.getIdCustomer());
        System.out.println("Nama Customer: " + customer.getNama());
        lihatKeranjang();
    }
}
```